EX: 9 Temporal Mining

Aim

To analyze time-based data and discover temporal patterns using **time-series analysis and moving average technique**.

Algorithm

1. Import required libraries (Pandas, NumPy, Matplotlib).
2. Load or create a time-based dataset.
3. Convert the date column into datetime format.
4. Set the date column as index.
5. Sort the data based on time.
6. Perform temporal analysis:
    - Calculate moving average
    - Identify trends over time
7. Visualize original data and temporal patterns.
8. Interpret the results.

Code

```
# 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt


# 2. Create Sample Time-Series Data
data = {
    'date': pd.date_range(start='2023-01-01', periods=30, freq='D'),
    'sales': np.random.randint(100, 500, size=30)
}
```

```
df = pd.DataFrame(data)

# 3. Convert to datetime and set index
df['date'] = pd.to_datetime(df['date'])
df.set_index('date', inplace=True)

# 4. Sort data
df = df.sort_index()

# 5. Temporal Mining (Moving Average)
df['moving_avg'] = df['sales'].rolling(window=5).mean()

# 6. Visualization
plt.figure(figsize=(10,5))
plt.plot(df.index, df['sales'], label='Original Sales')
plt.plot(df.index, df['moving_avg'], label='Moving Average', linestyle='--')

plt.title("Temporal Mining - Sales Trend Analysis")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.show()

# 7. Display Data
print(df.head())
```

Result

- The time-series data was successfully analyzed.
- The **moving average** smoothed fluctuations and revealed the overall trend